

Vending Machine Management System

Design Document | Program Design & Development

1. Terminology & Formulae

Term	Meaning
Customer	The person buying items. Identified by a phone number entered at the start of a session; the name and history persist under that number.
Phone Number	Unique text identifier for a customer. Used to look up an existing customer or create a new one.
Product	An item for sale, grouped into cold beverages, hot beverages, or snacks, with a selling price and an owner-facing cost.
Sellable Quantity	A product's current stock is available for purchase after removing expired or damaged units.
Stock Batch	One dated addition to a product's stock: a quantity and an expiry date. Oldest batches are sold first.
Restock Threshold	Owner-configured sellable quantity at or below which a product appears in Restock Alerts.
Warning Threshold	Owner-configured sellable quantity at or below which the customer sees an 'Only N left!' message.
Cart	The customer's in-progress selection of products and quantities for the current session; discarded after checkout or exit.
Cart Item	One line in the cart: a product and a quantity.
Coupon	An owner-defined rule that reduces a cart's total by a percentage, subject to a minimum order and eligibility rules.
Loyalty Points	Points a customer earns on completed purchases and may be redeemed toward a future purchase.
Order	A saved record of one completed payment. Also displayed back to the customer as the receipt.
Stock Removal	An owner-recorded reduction of sellable stock—damaged, expired, or package damage—outside of a sale.
Return	A customer requested to send back a purchased item because it was damaged or expired, verified against the original order.
Owner Password	A single password set on first use and required on every later session to access Owner functions.

Formulas—Cart & Coupons

$\text{Cart Total} = \sum (\text{item.price} \times \text{item.qty})$
 $\text{Coupon Discount} = \text{round}(\text{Cart Total} \times \text{coupon.percent_off} / 100)$ [only if $\text{Cart Total} \geq \text{coupon}$].

```
min_order]
Shortfall = coupon.min_order - Cart Total [shown when Cart Total < minimum order]
```

Formulas—Loyalty

```
Loyalty Points Earned = floor(Final Total / 10) [1 point per Rs 10 spent]
Loyalty Redemption Value = points redeemed × Rs-per-point [configured redemption rate]
```

Formulas—Payment

```
Final Total = Cart Total - Coupon Discount - Loyalty Redemption Value
Change = Amount Paid - Final Total
```

Formulas—Returns & Loss

```
Refund = original unit price × quantity returned [price taken from the order, not the current catalog price]
Stock Loss (per removal) = quantity × product cost
```

Formula—Restock Alert

```
Restock Alert triggers when Sellable Quantity ≤ Restock Threshold.
```

Formula—Profit & Loss

```
Sales Revenue = Σ order.final_total (all completed orders)
Product Costs = Σ (item.qty × product.cost) (all sold items)
Refunds = Σ return.refund_amount
Stock Losses = Σ removal.qty × product.cost (Damaged + Expired + Package Damage)

Net Profit/Loss = Sales Revenue - Product Costs - Refunds - Stock Losses
```

The coupon discount is not subtracted again here—it is already reflected inside sales revenue, since the order. `final_total` is recorded post-discount.

2. Program Flow

1. Program starts; `load_data()` restores PRODUCTS, CUSTOMERS, COUPONS, ORDERS, STOCK_REMOVALS, RETURNS, and OWNER_AUTH from data. JSON builds the default products and default coupons (WELCOME40, SAVE20, SAVE10, SAVE5) on the first run.
2. **The** entry menu is displayed: 1. Continue as Customer, 2. Continue as Owner, 3. Exit.
3. Customer path: prompt for name and phone; `get_or_create_customer` looks up or creates the customer record; display the customer menu.
4. Customer menu choices 1–11 route to the matching logic. py function (View Products, Select Product, Add to Cart, View Cart, Edit Cart Item, Apply/Remove Coupon, View Loyalty Points, Make Payment, View Receipt, View Order History, View Customer Report); after each, redisplay the Customer Menu.
5. Customer Menu choice 12—Exit: return to the entry menu (step 2).
6. Owner path: if `OWNER_AUTH.is_set` is False, prompt to create and confirm a password; otherwise prompt for the saved password. An incorrect password displays 'Incorrect password' and returns to the entry menu. On success, display the Owner Menu.
7. Owner menu choices 1–7 route to the matching logic. py function (Add Stock, Remove Stock, View Inventory, View Sales, View Reports, Manage Coupons, View Restock Alerts); after each, redisplay the Owner Menu.
8. Owner Menu choice 8—Logout: call `save_data()`; return to the entry menu (step 2).

9. Any invalid choice at any menu: display 'Invalid choice' and redisplay the current menu.
10. Entry menu choice 3—Exit: call `save_data()`; display 'Goodbye!'; terminate the program.

3. Classes & Function Signatures

Classes used

- **Product**—represents one item for sale; stores `product_id`, `name`, `category`, `price`, `cost`, `sellable_qty`, `restock_threshold`, `warning_threshold`, and its own list of `StockBatch` objects.
- **StockBatch**—stores `batch_id`, `qty`, `expiry_date`, and `added_at` for one addition of stock to a product.
- **CartItem**—stores `product_id` and `qty` for one line of the current cart.
- **Customer**—stores `phone`, `name`, `is_first_time`, `loyalty_points`, and `order_ids`.
- **Coupon**—stores `code`, `percent_off`, `min_order`, `first_time_only`, `enabled`, `usage_limit_per_customer`, and `used_by`.
- **Order**—stores the full record of one completed payment (also the receipt): `order_id`, `customer_phone`, `timestamp`, `items`, `coupon_code`, `coupon_discount`, `loyalty_redeemed`, `amount_paid`, `change`, and `final_total`.
- **StockRemoval**—stores `product_id`, `qty`, `reason`, `expiry_date`, `description`, and `timestamp` for one owner-recorded loss.
- **Return**—stores `order_id`, `product_id`, `qty`, `reason`, `expiry_date`, `description`, `refund_amount`, and `timestamp` for one customer return.
- **OwnerAuth**—stores `password_hash` and `is_set`.
- **CheckoutRequest**—stores the loyalty points to redeem and the cash tendered, gathered right before payment.
- **ReturnRequest**—stores order ID, product ID, quantity, reason, and (when applicable) expiry date or description, gathered before a return is validated.
- **PurchaseResult**—stores the values needed to display a receipt after checkout or an error message.
- **ReturnResult**—stores the refund and confirmation after a return is processed, or an error message.

main.py

Function	Purpose
<code>display_entry_menu()</code> -> None	Print the entry menu (Customer / Owner / Exit).
<code>get_menu_choice()</code> -> str	Read one line of raw menu input.
<code>prompt_customer_identity()</code> -> tuple[str, str]	Ask for name and phone number.
<code>display_customer_menu(customer: Customer)</code> -> None	Print the customer's name and the 12 customer menu options.
<code>display_owner_menu()</code> -> None	Print the 8 owner menu options.
<code>prompt_owner_password(is_setup: bool)</code> -> str tuple[str, str]	Ask for the password, and its confirmation when <code>is_setup</code> is True.
<code>prompt_product_selection()</code> -> tuple[str, int]	Ask for a category/product and a quantity to add to the cart.

Function	Purpose
<code>prompt_cart_edit()</code> -> tuple[str, int]	Show the cart; ask which line to edit and the new quantity.
<code>prompt_coupon_choice(available: list[Coupon])</code> -> str	Print each available coupon with its code, percentage, and minimum order (or shortfall); read the choice.
<code>prompt_checkout_inputs()</code> -> CheckoutRequest	Ask whether to redeem loyalty points, how many, and the cash tendered; return a CheckoutRequest.
<code>prompt_stock_addition()</code> -> tuple[str, int, int, str]	Ask for product, quantity, cost, and expiry date (Owner: Add Stock).
<code>prompt_stock_removal()</code> -> tuple	Ask for product, quantity, and reason; then expiry date (expired) or description (damaged/package damage).
<code>prompt_return_inputs()</code> -> ReturnRequest	Ask for the order ID, product, quantity, reason, and expiry date/description; return a ReturnRequest.
<code>prompt_report_choice()</code> -> str	Print the seven report options and read the choice.
<code>display_receipt(result: PurchaseResult)</code> -> None	Print the formatted receipt or the error if checkout failed.
<code>display_return_confirmation(result: ReturnResult)</code> -> None	Print the refund and confirmation, or the error if the return failed.
<code>display_inventory(products: list[Product])</code> -> None	Print each product's name, price, sellable quantity, and stock status.
<code>display_restock_alerts(products: list[Product])</code> -> None	Print each product at or below its restock threshold.
<code>display_report(report_name: str, rows: list)</code> -> None	Print a report title followed by its rows.
<code>display_error(message: str)</code> -> None	Print an error message as given.
<code>wait_for_enter()</code> -> None	Print 'Press Enter to return to menu' and block.
<code>main()</code> -> None	Load data; run the entry/customer/owner loops; save data before returning to the entry menu and on exit.

logic.py — Identity & Session

Function	Purpose
<code>get_or_create_customer(phone: str, name: str)</code> -> Customer	Search CUSTOMERS by phone; return the match, or create, store, and return a new customer.

logic.py — Catalog & Cart

Function	Purpose
<code>list_products(category: str None)</code> -> list[Product]	Return products, optionally filtered by category, flagged with a low-stock warning.
<code>add_to_cart(cart: list[CartItem], product_id: str, qty: int)</code> -> None	Validate quantity and available stock; append or increase a CartItem.

Function	Purpose
<code>edit_cart_item(cart: list[CartItem], product_id: str, new_qty: int) -> None</code>	Re-check current stock and update or remove the CartItem.
<code>get_cart_total(cart: list[CartItem]) -> int</code>	Sum price × qty across cart items.

logic.py — Coupons

Function	Purpose
<code>list_available_coupons(customer: Customer, cart_total: int) -> list[Coupon]</code>	Filter by enabled/eligibility/usage limit; attach a shortfall to coupons below minimum order.
<code>apply_coupon(cart_total: int, code: str, customer: Customer) -> int</code>	Revalidate eligibility and minimum order; return the discount.
<code>create_coupon / update_coupon / set_coupon_enabled / delete_coupon(...) -> None</code>	Owner CRUD over COUPONS.

logic.py — Loyalty

Function	Purpose
<code>calculate_loyalty_earned(final_total: int) -> int</code>	Compute points earned on a completed purchase.
<code>resolve_loyalty_coupon_combination(coupon_applied: bool, points_requested: int) -> int</code>	Apply the project's configured combination rule; return the redemption allowed.
<code>redeem_loyalty_points(customer: Customer, points: int) -> int</code>	Validate balance, deduct points, and return the rupee value redeemed.

logic.py — Payment & Receipts

Function	Purpose
<code>checkout(customer: Customer, cart: list[CartItem], request: CheckoutRequest) -> PurchaseResult</code>	Validate expiry and payment, update stock, create an order, award loyalty points, and return a PurchaseResult.
<code>get_order(order_id: str) -> Order None</code>	Look up ORDERS by ID.
<code>get_orders_for_customer(phone: str) -> list[Order]</code>	Return every order for a given phone number.

logic.py — Inventory & Restock

Function	Purpose
<code>add_stock(product_id: str, qty: int, cost: int, expiry_date: str) -> None</code>	Append a new StockBatch and increase sellable_qty.
<code>remove_stock(product_id: str, qty: int, reason: str, expiry_date: str None, description: str None) -> None</code>	Validate available quantity; decrease sellable_qty; append a StockRemoval.
<code>expire_batches() -> None</code>	Move any batch past its expiry_date out of sellable_qty; record a StockRemoval with reason 'Expired.'

Function	Purpose
<code>get_restock_alerts()</code> -> list[Product]	Return products at or below their restock threshold.

logic.py — Returns & Loss

Function	Purpose
<code>process_return(request: ReturnRequest) -> ReturnResult</code>	Verify the order/quantity/reason, compute the refund, record the return and matching stock removal, and return a ReturnResult.

logic.py — Reports

Function	Purpose
<code>sales_report()</code> -> list	List recorded orders with items, coupons, loyalty redemption, and final amounts.
<code>inventory_report()</code> -> list	List every product with its sellable quantity and stock status.
<code>damaged_report()</code> / <code>expired_report()</code> -> list	List STOCK_REMOVALS filtered by reason 'Damaged'/'Package Damage' or 'Expired.'
<code>return_report()</code> -> list	List all recorded RETURNS with refund amounts.
<code>low_stock_report()</code> -> list	Alias of <code>get_restock_alerts</code> formatted for the Reports menu.
<code>profit_loss_report()</code> -> dict	Compute sales revenue, product costs, refunds, stock losses, and the net profit/loss formula.
<code>customer_report(phone: str) -> dict</code>	Summarize a customer's order history and loyalty status.

logic.py — Persistence & Auth

Function	Purpose
<code>load_data()</code> -> None	Read data.json into the module-level structures, or initialize defaults on the first run.
<code>save_data()</code> -> None	Write the module-level structures back to data. JSON.
<code>verify_owner_password(password: str) -> bool</code>	Compare against OWNER_AUTH's stored hash.
<code>set_owner_password(password: str) -> None</code>	Hash and store a new owner password; set <code>is_set</code> to True.

4. Data Structures

Classes Product, StockBatch, CartItem, Customer, Coupon, Order, StockRemoval, Return, OwnerAuth, CheckoutRequest, ReturnRequest, PurchaseResult, and ReturnResult are used to group related data and, where relevant, keep the fields a function needs together as a single argument or return value.

Lists: PRODUCTS is stored as a list of Product objects rather than a dictionary; each Product carries its own list of StockBatch objects as an attribute, so a single lookup reaches both a product and its batches. The cart is a list of CartItem objects, rebuilt each session and discarded after checkout or exit. CUSTOMERS and COUPONS are

likewise lists, searched linearly by phone number or coupon code—the dataset is small enough that no separate index is needed. ORDERS, STOCK_REMOVALS, and RETURNS are append-only lists: nothing looks an entry up by position, so reports simply iterate the whole list and filter or sum as needed.

Lists are traversed for finding a product or customer by ID, selecting the oldest non-expired batch to draw stock from (FIFO), filtering coupons down to what a specific customer is eligible for, checking a return against its original order, and sorting products for display.

5. Function-Level Algorithms

main.py

- `display_entry_menu`—print the title and the three entry options.
- `get_menu_choice` — read and return one line of input.
- `prompt_customer_identity`—ask for name and phone; return both.
- `display_customer_menu`—print the customer's name and the 12 numbered options.
- `display_owner_menu`—print the 8 numbered options.
- `prompt_owner_password`—if `is_setup`, ask for the password and its confirmation and return both; otherwise, ask once and return it.
- `prompt_product_selection`—display categories, ask for a product and a quantity; return both.
- `prompt_cart_edit`—display the cart, ask which line and the new quantity; return both.
- `prompt_coupon_choice`—print each available coupon with its code, percentage, and minimum order (or shortfall); read and return the chosen code, or 0.
- `prompt_checkout_inputs`—ask whether to redeem loyalty points, how many, and the cash tendered; build and return a `CheckoutRequest`.
- `prompt_stock_addition`—ask for product, quantity, cost, and expiry date; return the four values.
- `prompt_stock_removal`—ask for product, quantity, and reason; ask for an expiry date if expired, or a description if damaged/package damage.
- `prompt_return_inputs`—ask for order ID, product, quantity, and reason; ask for an expiry date or description accordingly; build and return a `ReturnRequest`.
- `prompt_report_choice`—print the seven report options and read the choice.
- `display_receipt`—read `PurchaseResult` fields and print items, coupon, loyalty redemption, amount paid, change, and final total in the PRD's receipt format.
- `display_return_confirmation`—print the refund and confirmation, or the `ReturnResult`'s error message.
- `display_inventory`—print each product's name, price, sellable quantity, and low-stock flag.
- `display_restock_alerts`—print each product returned by `get_restock_alerts` with a RESTOCK ALERT message.
- `display_report`—print the report name as a header, then each row.
- `display_error`—print the message as given.
- `wait_for_enter`—print the prompt and block until Enter.
- `main`—call `load_data()`; loop over the entry menu, dispatching to the Customer or Owner loop; call `save_data()` before returning to the entry menu and on exit.

logic.py — Identity & Session

- `get_or_create_customer`—search CUSTOMERS by phone; if found, return it as stored; otherwise create, append, and return a new Customer with `is_first_time=True`.

logic.py — Catalog & Cart

- `list_products` — filter PRODUCTS by category if given; attach an ‘Only N left!’ flag when `sellable_qty ≤ warning_threshold`.
- `add_to_cart`—reject a non-positive or non-integer quantity; reject if qty exceeds the product's `sellable_qty`; otherwise append a new CartItem or increase an existing one.
- `edit_cart_item`—re-read the product's current `sellable_qty`; if the new quantity fits, update or remove the CartItem; otherwise leave it unchanged and raise `InsufficientStockError`.
- `get_cart_total` — sum `price × qty` across all CartItems.

logic.py — Coupons

- `list_available_coupons`—keep enabled coupons; drop `first_time_only` coupons for returning customers or customers already in `used_by`; attach a shortfall value to coupons below the cart's minimum order.
- `apply_coupon`—revalidate the same eligibility rules as `list_available_coupons` at the moment of applying; return `round(cart_total × percent_off / 100)`, or raise `InvalidCouponError`.
- `create_coupon` / `update_coupon` / `set_coupon_enabled` / `delete_coupon`—validate the owner's input and add, modify, enable/disable, or remove the matching coupon in COUPONS.

logic.py — Loyalty

- `calculate_loyalty_earned` — `floor(final_total / 10)`.
- `resolve_loyalty_coupon_combination`—apply the project's configured rule (Section 8 of the earlier discussion) to decide whether/how much redemption is allowed alongside an applied coupon.
- `redeem_loyalty_points`—validate the customer has enough points, subtract them, and return the rupee value redeemed.

logic.py — Payment & Receipts

- `checkout`—re-check that each cart item's product hasn't expired since being added; reject if payment is insufficient; otherwise decrement `sellable_qty` from the oldest non-expired batch first (FIFO); build and append a new Order; call `calculate_loyalty_earned` and add the points to the customer; return a `PurchaseResult` built from the new Order.
- `get_order` — linear search of ORDERS by `order_id`.
- `get_orders_for_customer` — filter ORDERS where `customer_phone` matches.

logic.py — Inventory & Restock

- `add_stock`—append a new StockBatch to the product's batch list and increase `sellable_qty` by the same amount.
- `remove_stock`—reject if qty exceeds `sellable_qty`; otherwise decrease `sellable_qty` and append a `StockRemoval` with the given reason/expiry/description.
- `expire_batches`—for every batch past its `expiry_date` still counted in `sellable_qty`, subtract its quantity and append a `StockRemoval` with reason ‘Expired’; run this at the top of every menu render.
- `get_restock_alerts` — filter PRODUCTS where `sellable_qty ≤ restock_threshold`.

logic.py — Returns & Loss

- `process_return`—find the order and confirm the product/quantity are covered by it minus any prior returns; require an `expiry_date` for Expired or a description for Damaged; compute the refund from the order's recorded unit price; append a Return and a matching StockRemoval; return a `ReturnResult`.

logic.py — Reports

- `sales_report` / `inventory_report` / `damaged_report` / `expired_report` / `return_report` / `low_stock_report`—each filters/aggregates the relevant list (ORDERS, PRODUCTS, STOCK_REMOVALS, RETURNS) into display rows.
- `profit_loss_report`—sum ORDERS for revenue, sold-item cost for product costs, RETURNS for refunds, and STOCK_REMOVALS for stock losses; compute Net Profit/Loss per Section 1's formula.
- `customer_report`—pull the customer's loyalty points and their orders via `get_orders_for_customer`.

logic.py — Persistence & Auth

- `load_data`—read `data.json` if it exists and populate PRODUCTS/CUSTOMERS/etc.; otherwise build default products and default coupons (WELCOME40/SAVE20/SAVE10/SAVE5) and an empty `OwnerAuth`.
- `save_data`—serialize all module-level lists back to data. JSON.
- `verify_owner_password`—hash the given password and compare it to OWNER_AUTH's stored hash.
- `set_owner_password`—hash and store a new password; set `is_set` to True.

6. Error Handling

Situation	Message	Result
Invalid product or non-integer/non-positive quantity	Please enter a valid product and a positive whole number.	Reject input; ask again.
Quantity exceeds sellable stock (add to cart).	Only {available} available.	Item not added.
Edited quantity exceeds current stock.	—	Reject the edit; keep the previous valid quantity.
Coupon invalid, disabled, ineligible, or below minimum order	{code} requires Rs {min_order}. Add Rs {shortfall} more to use {code}.	Coupon not applied.
The customer declines a coupon.	—	Continue directly to payment.
Payment less than final total	Payment is short by Rs {amount}.	Reject payment; ask again.
The product expires before final payment.	{product} is no longer available (expired).	Line item blocked; return to cart review.
Return cannot be verified, quantity is too high, or reason is not damaged/expired.	The return could not be verified.	Return rejected.
The owner removes more stock than available.	Cannot remove {qty}; only {available} in stock.	Removal rejected; inventory unchanged.
Owner password mismatch (login)	Incorrect password.	Access denied; return to entry menu.

Situation	Message	Result
Owner password confirmation mismatch (setup)	Passwords do not match.	Ask to set the password again.
Stock reaches restock threshold.	RESTOCK ALERT: {product} has reached its minimum stock level.	Product listed under Restock Alerts.
The customer has no previous orders or loyalty points.	No orders yet. / 0 points.	Show zero/empty state, not an error.
Invalid menu choice (any menu)	Invalid choice.	Redisplay the current menu.